

Lecture 20 – Passwords and Authentication

Stephen Checkoway

Oberlin College

Slides based on Bailey's ECE 422

AUTHENTICATION

Authentication Basics

- Authentication binds identity to a subject
- Two step process
 - Verification - process verifies and binds entity and identity (confirming who you are; e.g., do you own the email address you claim to own)
 - Identification - establish identity to system (confirming you are the identity you claim)

PASSWORD AUTHENTICATION

Basics

- User keeps a secret string (password)
- Something the user *knows*
- Advantages?
- Disadvantages?

Attacks

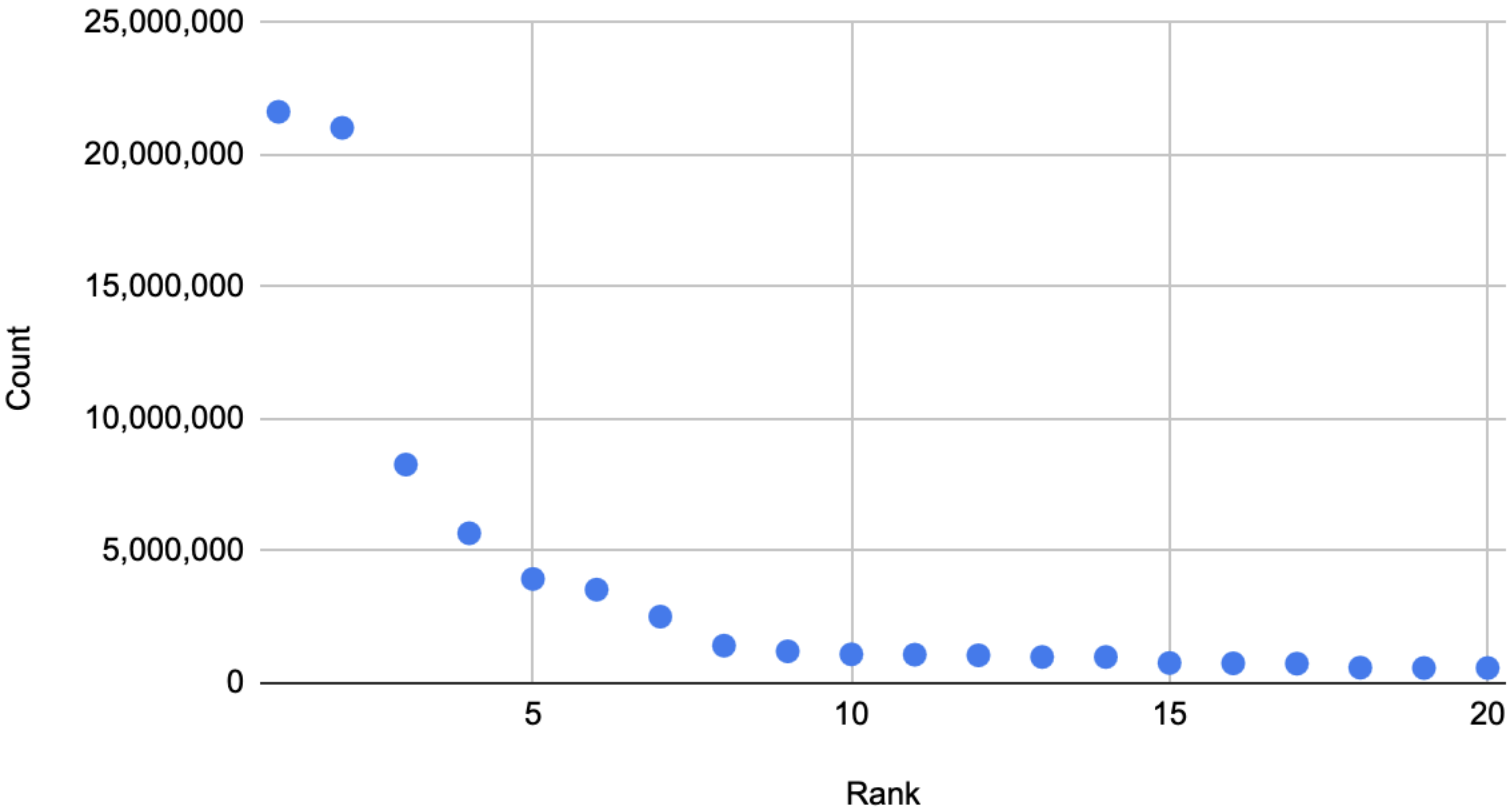
- Steal from the user
 - Install a keylogger (hardware or software)
 - Find it written down
 - Social engineering/Phishing
 - Intercept the password over network
 - Use a side channel
- Steal from the service
 - Install malware on the web server
 - Dump the password database with SQL injection
- Steal from a third party (password reuse)

Password Guessing

	PIN	Freq
#1	1234	10.713%
#2	1111	6.016%
#3	0000	1.881%
#4	1212	1.197%
#5	7777	0.745%
#6	1004	0.616%
#7	2000	0.613%
#8	4444	0.526%
#9	2222	0.516%
#10	6969	0.512%
#11	9999	0.451%
#12	3333	0.419%
#13	5555	0.395%
#14	6666	0.391%
#15	1122	0.366%
#16	1313	0.304%
#17	8888	0.303%
#18	4321	0.293%
#19	2001	0.290%
#20	1010	0.285%

Top 20 passwords (NordPass 2025)

Top 20 passwords



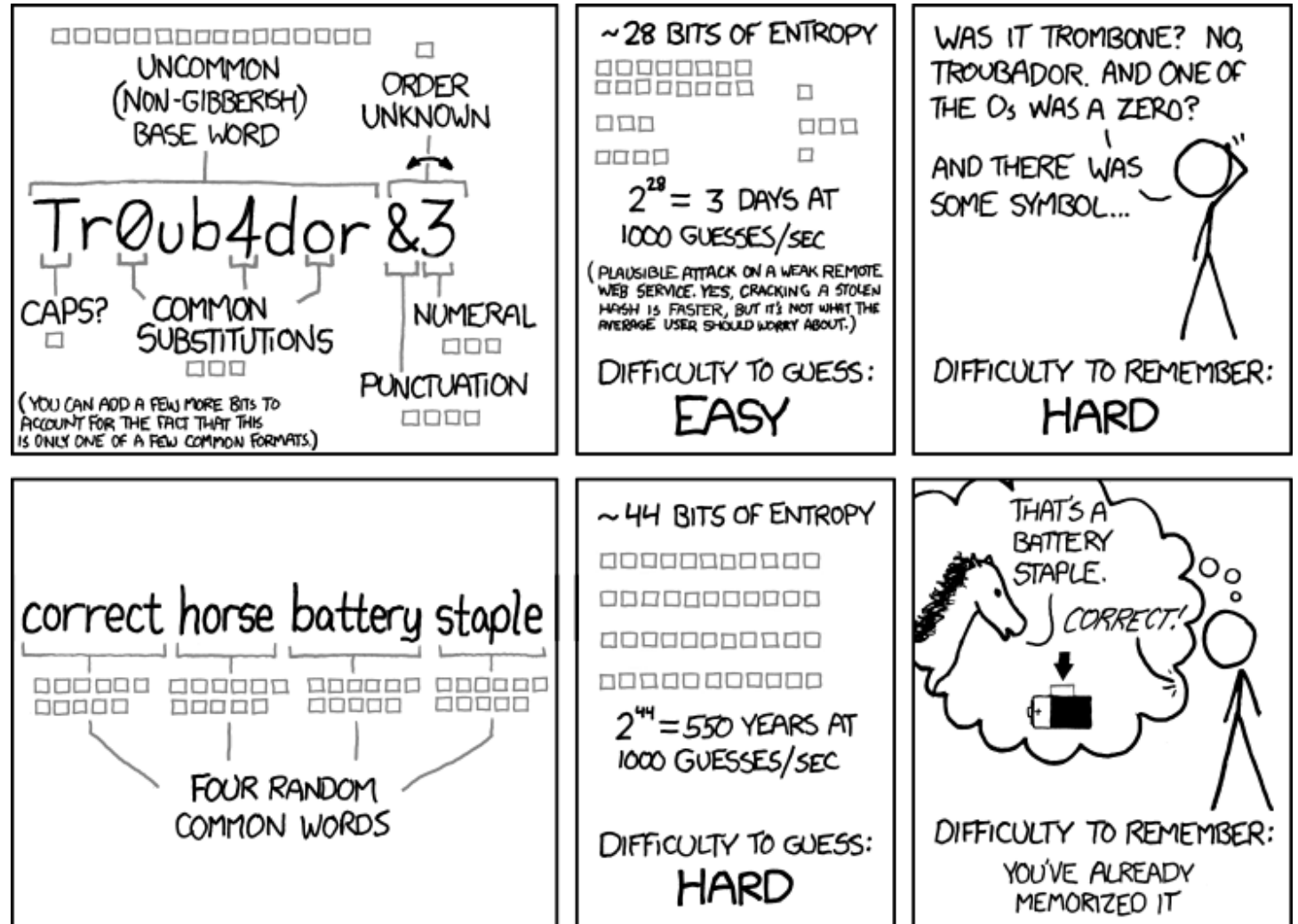
1	123456	21,627,656
2	admin	21,030,012
3	12345678	8,274,408
4	123456789	5,673,712
5	12345	3,950,777
6	password	3,545,119
7	Aa123456	2,520,728
8	1234567890	1,418,939
9	Pass@123	1,210,039
10	admin123	1,087,247
11	1234567	1,084,354
12	123123	1,060,563
13	111111	990,391
14	12345678910	988,396
15	P@ssw0rd	770,658
16	Password	755,709
17	Aa@123456	735,141
18	admintelecom	585,620
19	Admin@123	579,512
20	112233	576,908

Secure Passwords

- Uneven distribution makes guessing easier
- Passwords should be uniformly distributed
 - All characters in password chosen with equal probability
- Passwords should be long
 - Longer password = larger brute force search space
- Passwords should never be reused
- Passwords chosen randomly are difficult to remember
 - Tradeoff of security vs. convenience

Password Strength

<https://xkcd.com/936/>



THROUGH 20 YEARS OF EFFORT, WE'VE SUCCESSFULLY TRAINED EVERYONE TO USE PASSWORDS THAT ARE HARD FOR HUMANS TO REMEMBER, BUT EASY FOR COMPUTERS TO GUESS.

STORING PASSWORDS

Confirmed Attack At Opera, 1.7M Password Leak Possible



Epic Games forums hacked again: Over 800,000 gamers put at risk

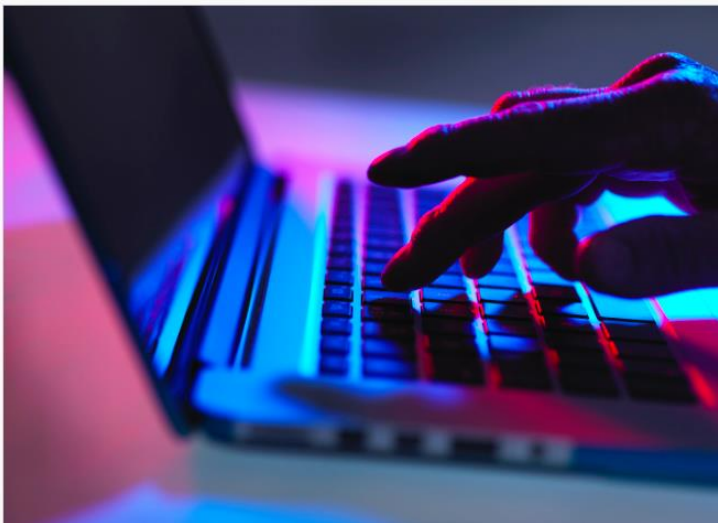
BY GRAHAM CLULEY POSTED 23 AUG 2016 - 02:50AM

DATA LEAKAGE



43 million passwords hacked in Last.fm breach

Posted Sep 1, 2016 by John Mannes (@JohnMannes)



Passwords for 32M Twitter accounts may have been hacked and leaked

Posted Jun 8, 2016 by Catherine Shu (@catherineshu), Kate Conger (@kateconger)



Next Story



CrunchBase

Twitter

FOUNDED
2006

OVERVIEW

Twitter is a global social networking platform that allows its users to send and read 140-character messages known as "tweets". It enables registered users to read and post their tweets through the web.

2016 mega breaches continue as hackers steal and leak 33 million QIP.ru accounts

Breach appeared to have occurred in 2011 and user passwords were allegedly not encrypted.



By India Ashok

September 10, 2016 11:52 BST



Hackers breach porn site, expose 800,000 user accounts

A massive data breach has invaded the popular porn repository Brazzers' sister site, Brazzers Forum, after hackers took control of the website with nearly 800,000 user account information, including usernames and passwords.

By Yves Matthew Amodia | Sep 13, 2016 09:55 AM EDT



TC NEWS

The Daily Crunch

Our top headlines
Delivered daily

CrunchBase Daily

The latest startup funding announcements
Delivered daily

Enter Address

Okay, but those are old

These are password
breaches from this year
alone

<https://haveibeenpwned.com/PwnedWebsites>

Breach Name	Pwn Count	Added to HIBP	Breach Date
Divine Skins	105.8k	15-Mar-2026	Mar-2026
Baydöner	1.3M	15-Mar-2026	Mar-2026
Provecho	712.9k	03-Mar-2026	Jan-2026
Lovora	495.6k	02-Mar-2026	Feb-2026
Quitbro	22.9k	02-Mar-2026	Feb-2026
KomikoAI	1.1M	02-Mar-2026	Feb-2026
Odido	6.1M	26-Feb-2026	Feb-2026
Canadian Tire	38.3M	25-Feb-2026	Oct-2025
CarGurus	12.5M	22-Feb-2026	Feb-2026
CarMax	431.4k	20-Feb-2026	Jan-2026
Figure	967.2k	18-Feb-2026	Jan-2026
Canada Goose	581.9k	17-Feb-2026	Jul-2025
University of Pennsylvania	623.8k	16-Feb-2026	Oct-2025
APOIA.se	450.8k	16-Feb-2026	Dec-2025
Toy Battles	1k	10-Feb-2026	Feb-2026
Association Nationale des Premiers Secours	5.6k	10-Feb-2026	Jan-2026

Storing Passwords

- Password database is highly sensitive
- We should ***never*** store *plaintext* passwords
- Store something that lets user prove they know the password

Hash functions (more later)

- Input – data of an arbitrary size
- Output – fixed length
- Same input always produces the same output
- One way function – cannot deduce input from output
- A “fingerprint” for the input
- Examples: ~~MD5, SHA-1~~, SHA-256, SHA-512, SHA-3
 - `md5 ("welcome") = 40be4e59b9a2a2b5dfffb918c0e86b3d7`
- ***None of these should be used directly used for password hashing***

Noncryptographic hash functions (and more)

- Cyclic redundancy checks (CRC)
 - CRC-16, CRC-32, etc.
 - Based on polynomials, many variants
- Checksums
 - sum-8, sum-16, Adler-32, Luhn algorithm, etc.
- Noncryptographic hash functions
 - FNV-1, Bernstein hash (djb2), Java's hashCode()
- ***None of these should be used for password hashing***

Don't use any of those

- All of those hash functions are designed to be as fast as possible
- Think about putting a string in a hash table, you want that to be a fast operation
- For passwords, we want the opposite: it should be slow and expensive to compute the hash [Why?]

Password Hashes

- We store a database of password *hashes*
- e.g., /etc/shadow on UNIX

```
rcunnin2:$6$vb1tLY1qiY$M.1ZCqKtJBxBtZm1gRi8B  
bkn39KU0YJW1cuMFzTRANcNKFKR4RmAQVk4rqQQCkaJT  
6wXqjUkFcA/qNxLyqW.U/:15405:0:99999:7:::
```

Password Cracking

- Brute force search through all possible passwords in order
- Use a dictionary
- Use a dictionary of common passwords
- Combine dictionary with common passwords and heuristics (e.g. p@\$\$w0rd and password123)
- Use statistical models of user passwords
- Easy to parallelize: hash password guess, compare to entire hash database
- Commonly done with arrays of GPUs

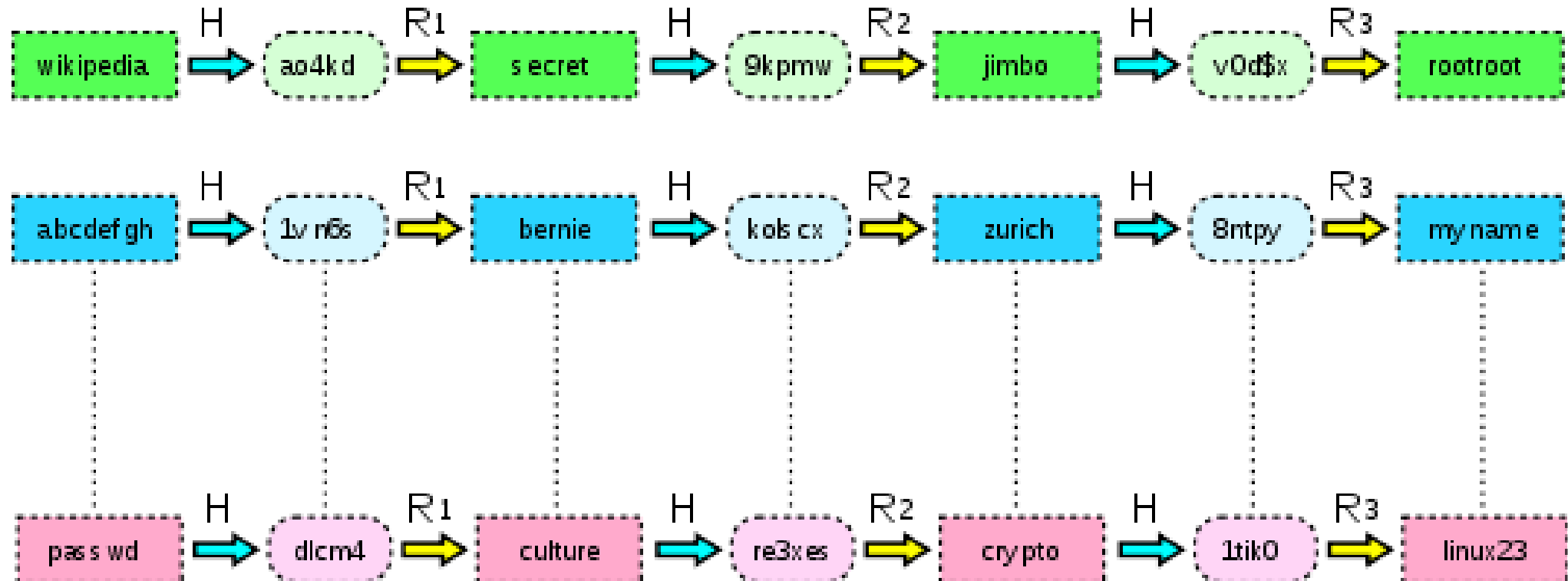
Precomputed tables of hashes

- Many passwords are common
- Precompute them in a table
- Too inefficient, table will be huge

Rainbow table (examples from Wikipedia)

- $H : P \rightarrow X$ is a hash function from the set of possible passwords P to hash values X
- $R_i : X \rightarrow P$ is a sequence of “reduction functions” from hash values to likely passwords
- Build the table by starting with a table of common passwords and compute a chain of length n by starting with H and alternating with the R_i in order

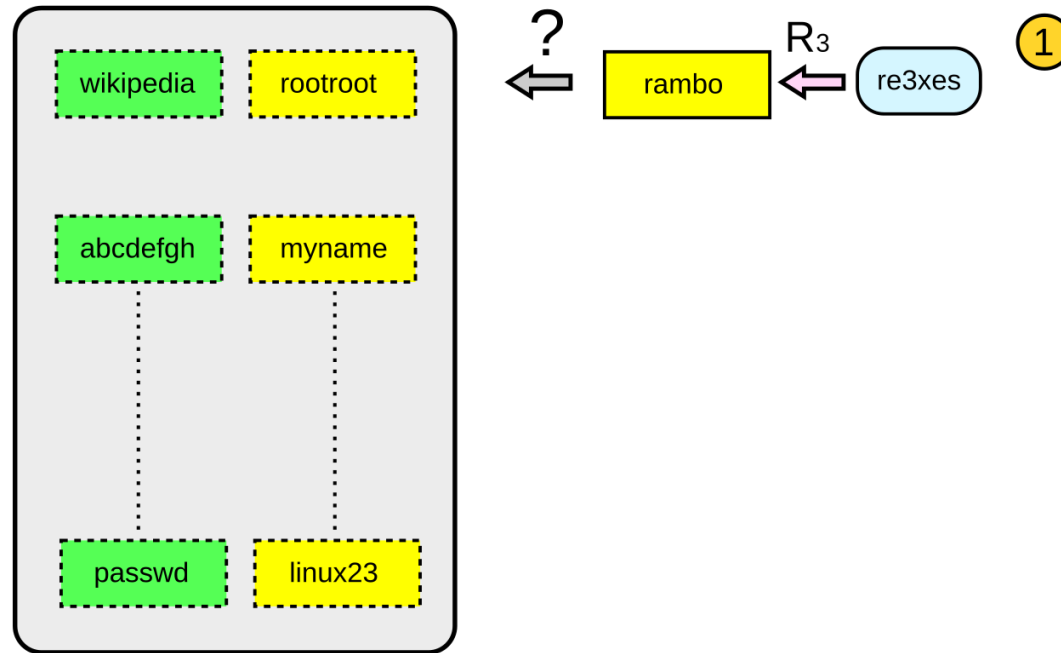
Alternating applying H and R_i



Only the start and end of each chain is stored in the table

Table lookup

1. Given a hash value x , compute $R_n(x)$ and check if it is one of the stored ends of a chain



Here, $x = \text{re3xes}$ is the initial hash; rambo is the computed value; it doesn't appear at the end of a chain

Table lookup

2. If not, compute $R_n \left(H \left(R_{n-1}(x) \right) \right)$ and check if is the end of a chain; repeat this step starting with successively earlier R_k

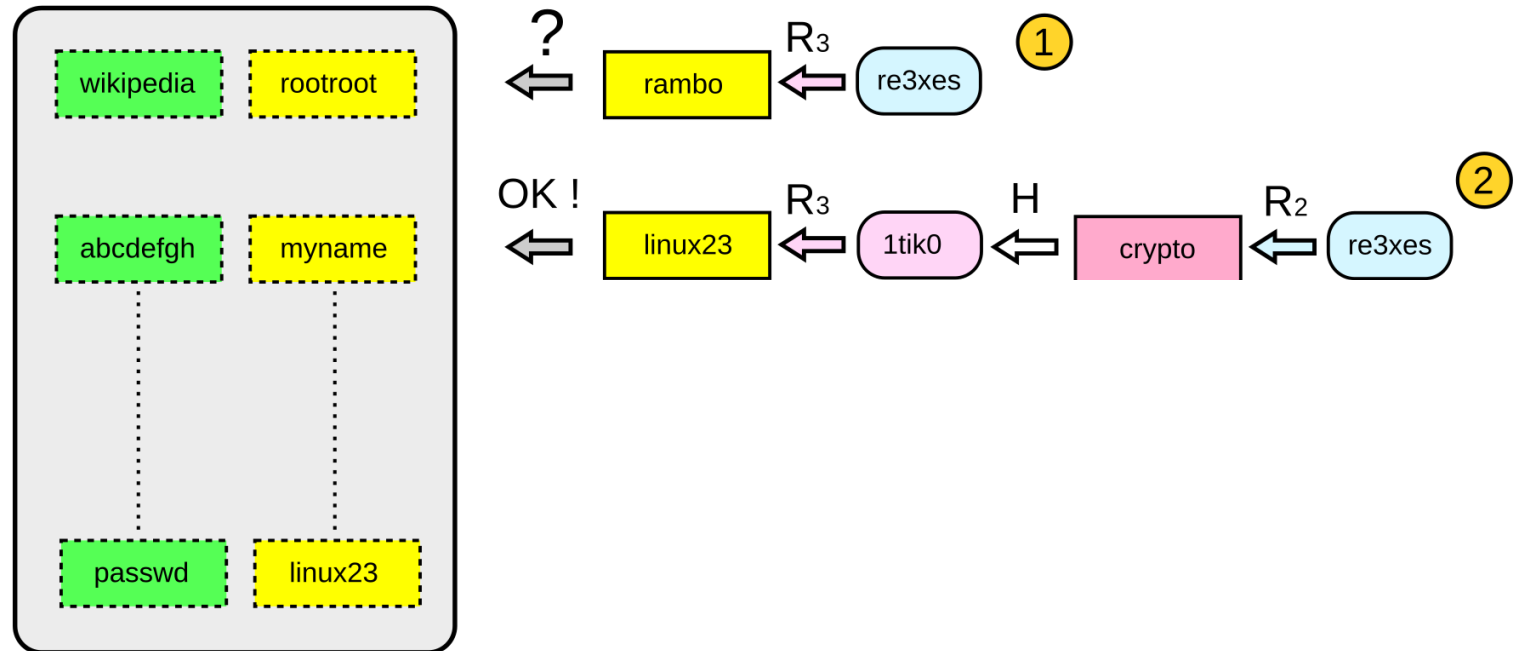
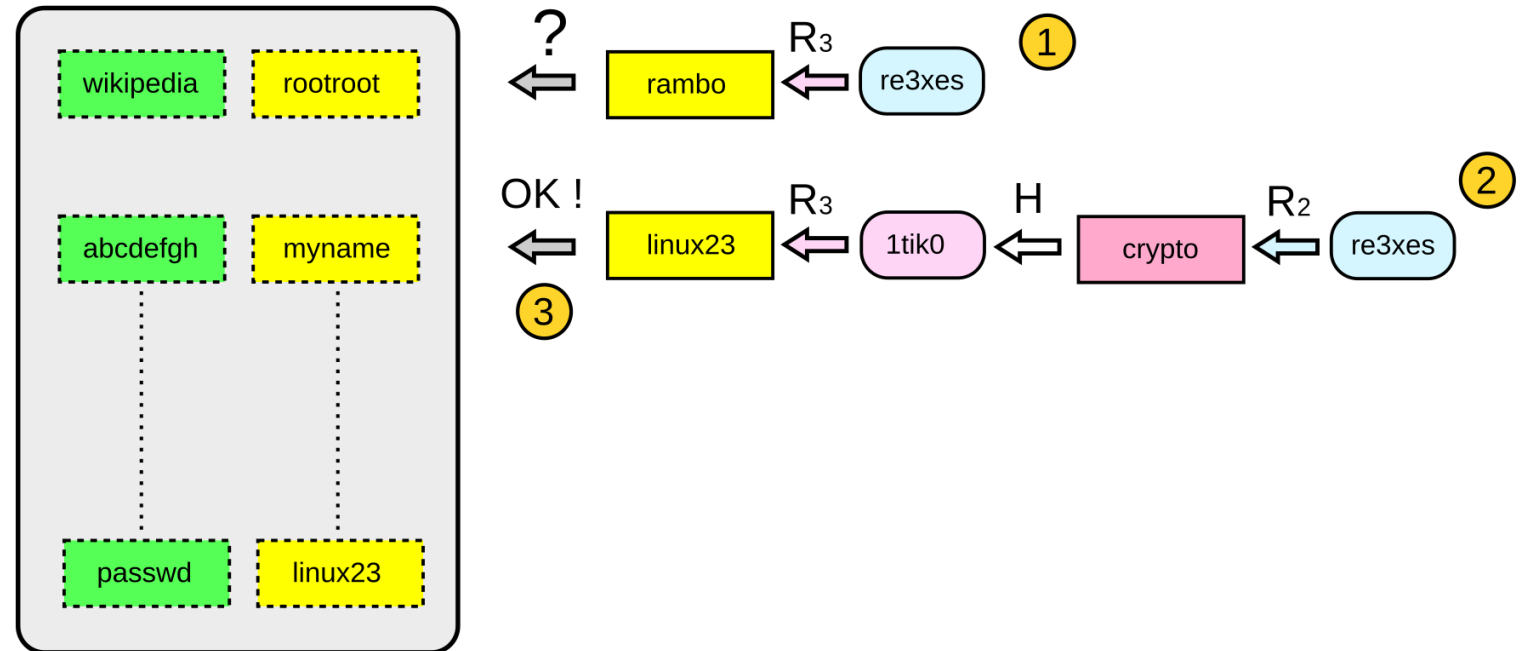


Table lookup

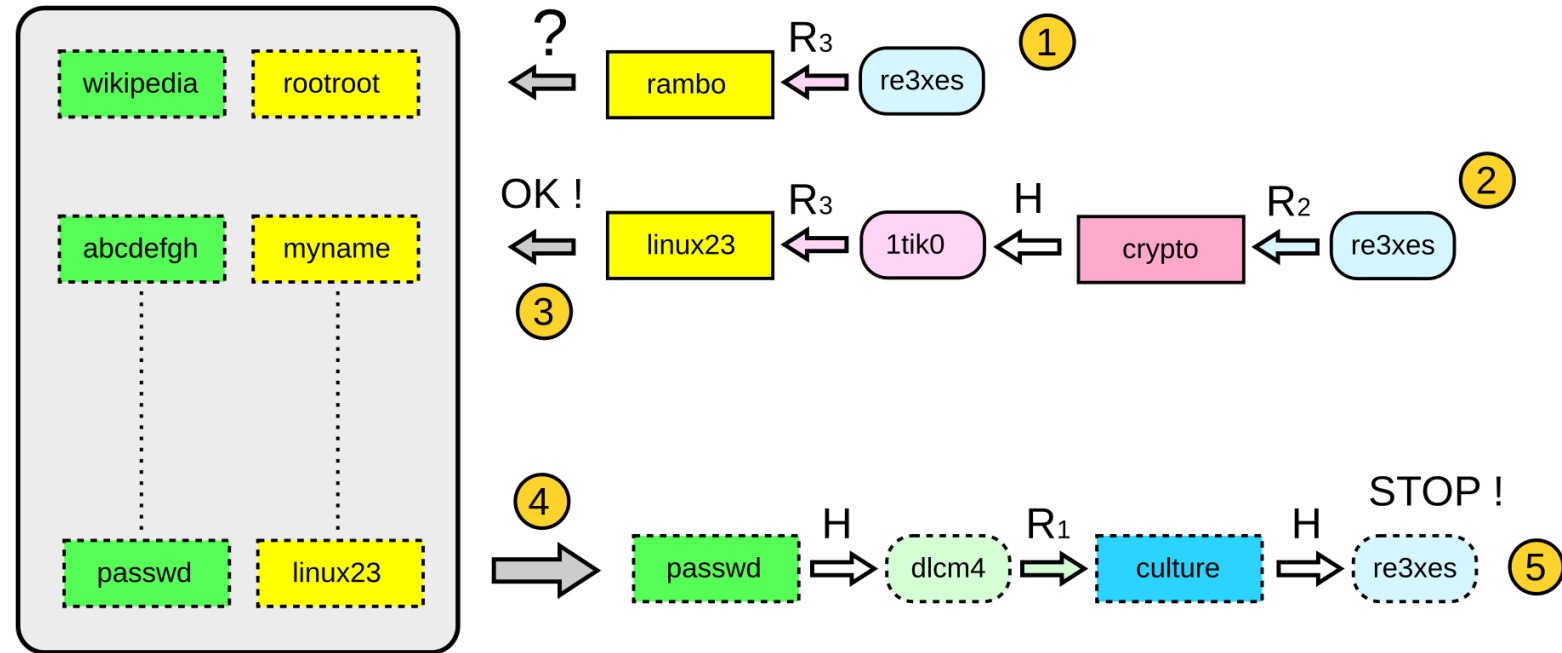
3. If the value from steps 1 or 2 is at the end of the chain, select the password from the start of the chain



Here, linux23 is the computed value that appears at the end of the chain starting with passwd; select passwd

Table lookup

- Starting with the selected password, generate the chain by alternating H and R_k
- Stop when the initial hash value is found



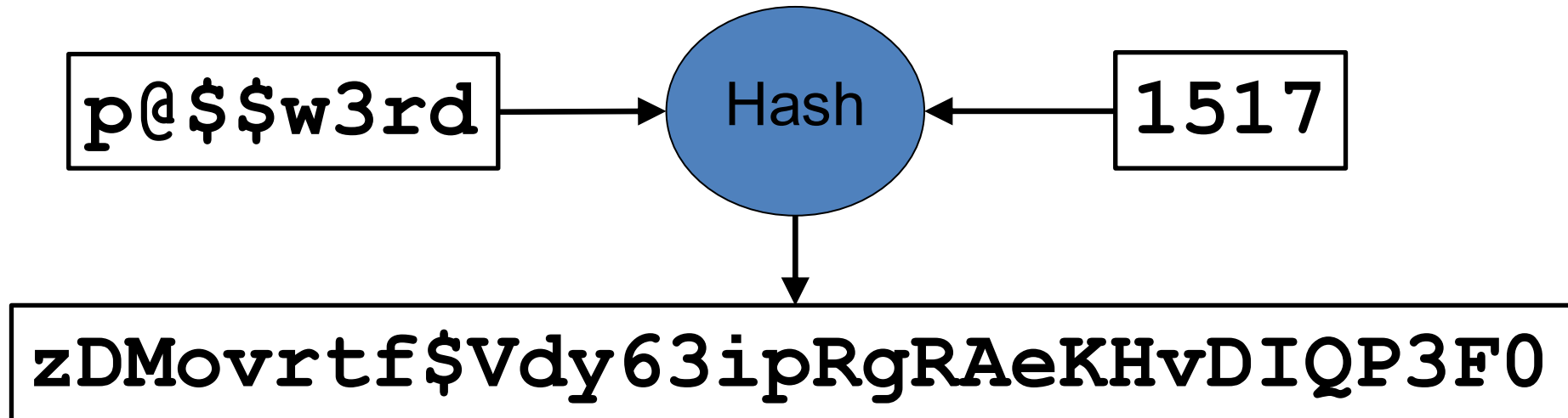
Starting with $H(\text{passwd})$, we eventually reach culture and $H(\text{culture}) = \text{re3xes}$ so culture is a password that has the right hash value

Rainbow table

- Time/space tradeoff by length of the hash chain: longer hashes require less space but longer lookups
- Rainbow tables work when a raw hash function (e.g., md5) is used to compute the hashes
- Defenders can defeat rainbow tables by using different hash functions for each password [How can we do that?]

Salting Password Database

- Generate and store a random number, the salt for each password
- Concatenate password and salt to compute hash
- Effectively a unique hash function for each password



Password Security Policies

- Educate users about password security
 - Specifically train them to use good passwords
 - But they might or might not follow through
- Generate passwords randomly
 - Perfect uniform distribution
 - But not very psychologically acceptable
- Reactive password checking
 - Crack your own user's passwords
 - But expensive and passwords vulnerable until cracked
- Complex password policy
- Proactive checking against databases like haveibeenpwned.com

Complex Password Policy

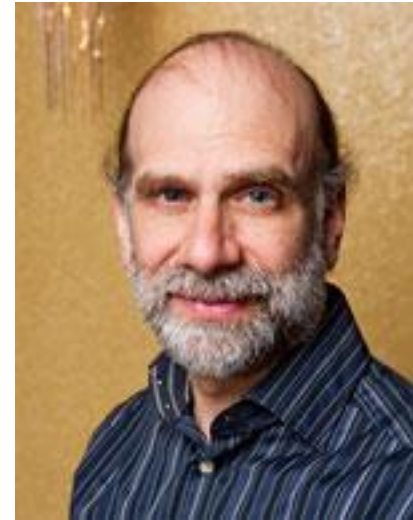
- Let the user select their own password
- Force them to follow a policy
- Reject passwords that don't follow policy
- But...
 - Technically *reduces* number of possible passwords
 - Policy might not be psychologically acceptable
 - We don't know if users are reusing their passwords

“Scunthorpe problem” but for usernames/passwords

- The Scunthorpe problem is when you block text because a word appears to contain something obscene (named after AOL’s profanity filter which prevented people in Scunthorpe, England from signing up)
- Sometimes complex password policies will block passwords apparently containing usernames
- My personal email address is s@...; sometimes I have to generate passwords that don’t contain an s to get around these policies

Security Questions

- Are also a shared secret
- Bruce Schneier calls them “a backup password”
- Easier to guess and social engineer
- Some cannot be changed
- Some websites have a fixed set of questions and answers! (E.g., United Airlines)



Breaches happen

- Databases of usernames and passwords are exposed
- <https://haveibeenpwned.com/> ← Use this!

THE VERGE

TECH ▾

SCIENCE ▾

CULTURE ▾

CARS ▾

REVIEWS ▾

LONGFORM

VIDEO

MORE ▾

US & WORLD

TECH

CYBERSECURITY

Yahoo says all 3 billion user accounts were impacted by 2013 security breach

by Natt Garun | @nattgarun | Oct 3, 2017, 5:07pm EDT

f SHARE

t TWEET

in LINKEDIN

RECENT PASSWORD SOLUTIONS

Password Managers

- Application that generates and maintains passwords
- Examples: Browsers, ~~LastPass~~, KeePass, DashLane, 1Password
- Advantages:
 - Can handle random passwords
 - **Can create unique passwords for every website and service**
- Disadvantages
 - One point of failure
 - Requires a strong password (could be snooped)
 - Could be hacked (only as secure as the password manager)
 - Can be inconvenient (doesn't work for some sites, set up time, etc.)

One Point of Failure...

Trend Micro password manager had remote command execution holes and dumped data to anyone: Project Zero

Google's Project Zero discovered multiple trivial remote code execution vulnerabilities sitting within a password manager installed by Trend Micro as default alongside its AntiVirus product



By [Chris Duckett](#) | January 12, 2016 -- 01:32 GMT (17:32 PST) | Topic:



A password management tool installed by default alongside T

KrebsonSecurity

In-depth security news and investigation

[HOME](#)[ABOUT THE AUTHOR](#)[ADVERTISING/SPEAKING](#)

Experts Fear Crooks are Cracking Keys Stolen in LastPass Breach

September 5, 2023

79 Comments

Single Sign-On (SSO)

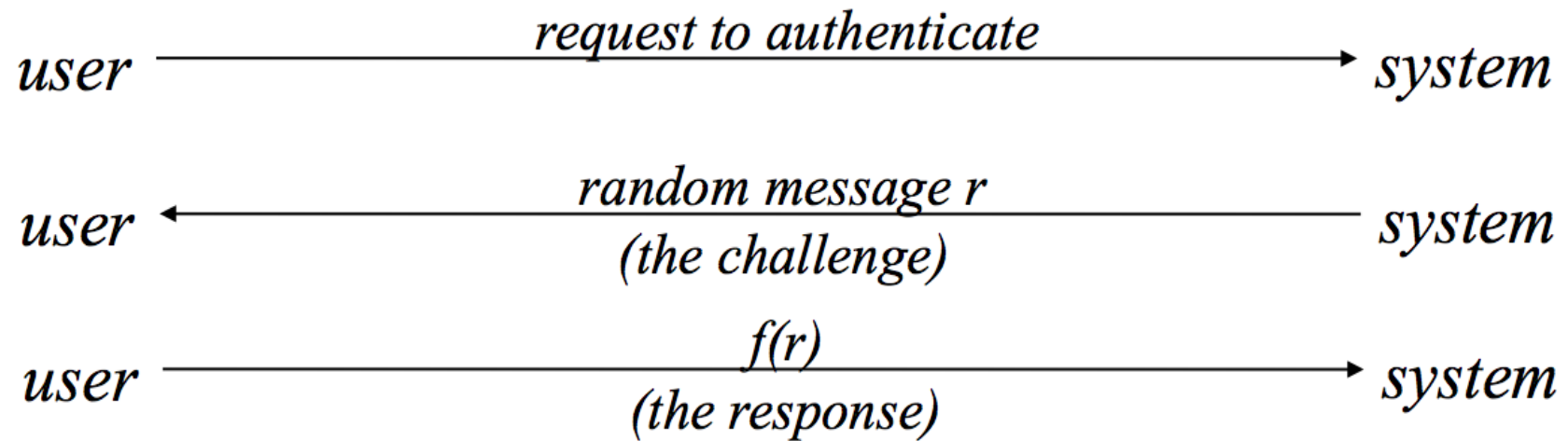
- Login to trusted 3rd party (identity provider), who vouches for user identity
 - Examples: Facebook Connect, OAuth, OpenID Connect
 - Pros and cons similar to Password Managers
 - Third party can track users...
-
- We use Okta for SSO

TOKEN-BASED AUTHENTICATION

Basics

- Something the user *has*
- Static memory cards
 - Read only
 - e.g., ATM card/Credit Card (using number + CVV2 or magstripe)
 - Vulnerable to replay attack
- Smart card
 - Storage and computation
 - Enables challenge-response or one-time password
 - Protects against replay attack

Challenge-Response



One-time password (OTP)

- Smart card can also implement one-time password scheme
- S/Key is one such scheme
- Time-based one-time password is another (TOTP)
- Vulnerable to man-in-the-middle (MitM)

S/Key

- Server
 - Generates random seed S
 - Computes $H(S)$, $H^2(S)$, $H^3(S)$, ..., $H^{n+1}(S)$ and gives the first n of them to the client in reverse order
 - Server stores $H^{n+1}(S)$ and discards the rest
- To authenticate
 - Client sends the first unused value P (initially $P=H^n(S)$) to the server
 - Server computes $H(P)$ and compares to the stored value
 - If there's a match, the client is authenticated and discards P ; the server discards the stored value and stores P in its place

TOTP

- The current time is turned into time-counter TC
 - $TC = \text{floor}(\text{now} - \text{start_time}) / \text{time_step}$
- TC and a secret key are used to create TOTP
 - $TOTP = \text{HOTP}(\text{secret_key}, TC)$
 - HOTP is based on a keyed-hash function
- The output is $TOTP \bmod 10^d$ for d digits
- Client and server both compute the output value based on the current time and the client sends it to the server

2 Factor Authentication (2FA)

- Something you **have** AND something you **know**
- Either factor is useless without the other
- Chip and PIN
- Commonly implemented in mobile phones via SMS
 - Disadvantages:
 - ONE device (if hacked)
 - SMS is easy to redirect
 - ONE point of failure for SE (phone company)
- Google authenticator, Duo Mobile, Authy, Yubico Authenticator
- OTP tokens (e.g., TOTP), U2F keys

Phishing

Phishing is an attempt to get users' account credentials

Usually an email with a link to click

Users log in, sending their user name and password to the server and thus the attacker

Phishing and 2FA using an OTP

Scenario: Attacker sends a phishing email with a link to a copycat website containing a login box with user/pass

The user's account is protected with 2FA where the second factor is a time-based OTP (TOTP)

Is the user's account protected or can the phisher still get the account credentials and log into the website as the user?

One-time Password MitM

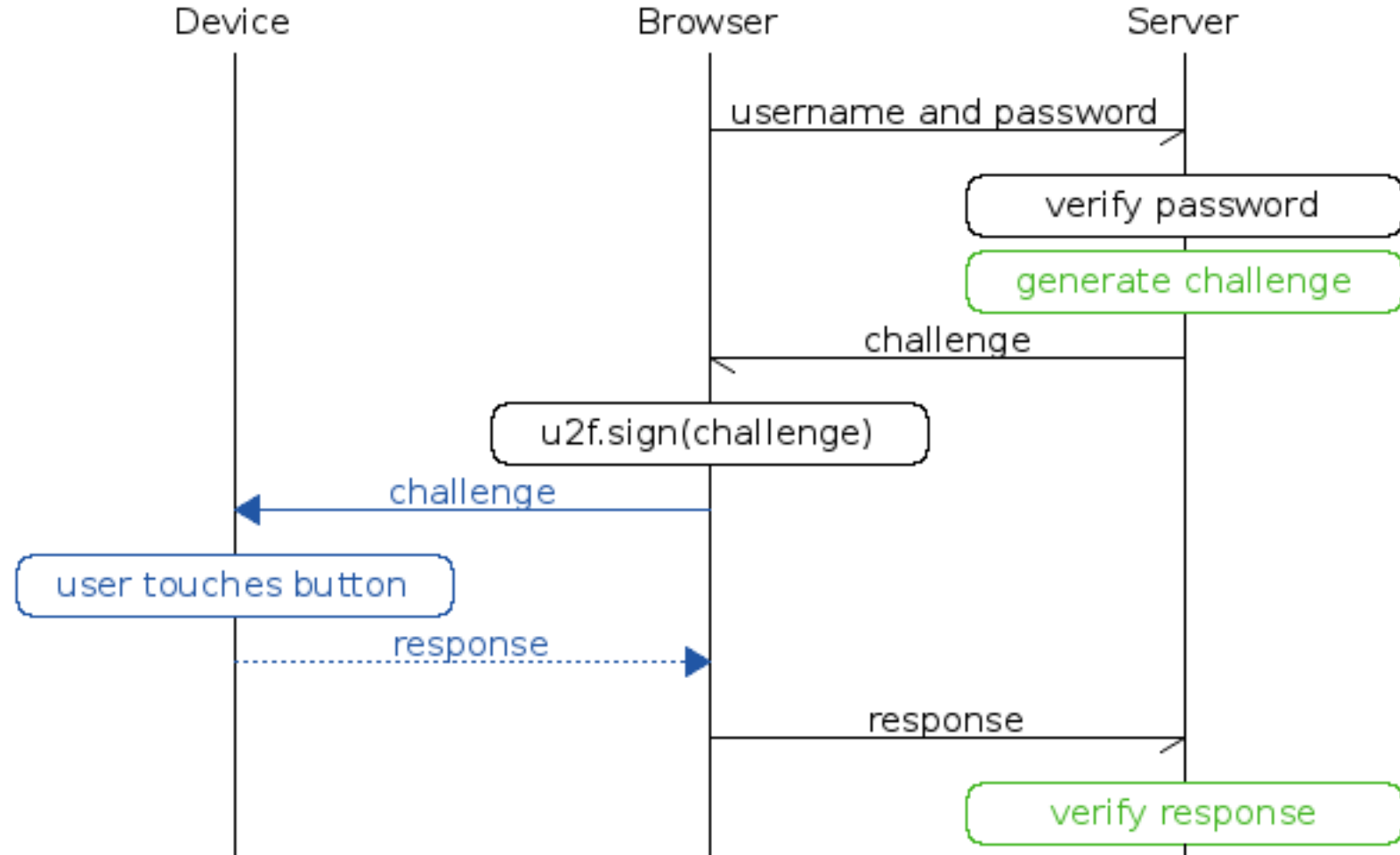
1. User sends username/password to phishing site
2. Phishing site forwards them to legitimate site
3. Legitimate site asks phisher for OTP
4. Phishing site asks user for OTP
5. User sends OTP to phisher
6. Phisher sends OTP to legitimate site

One-time Password MitM

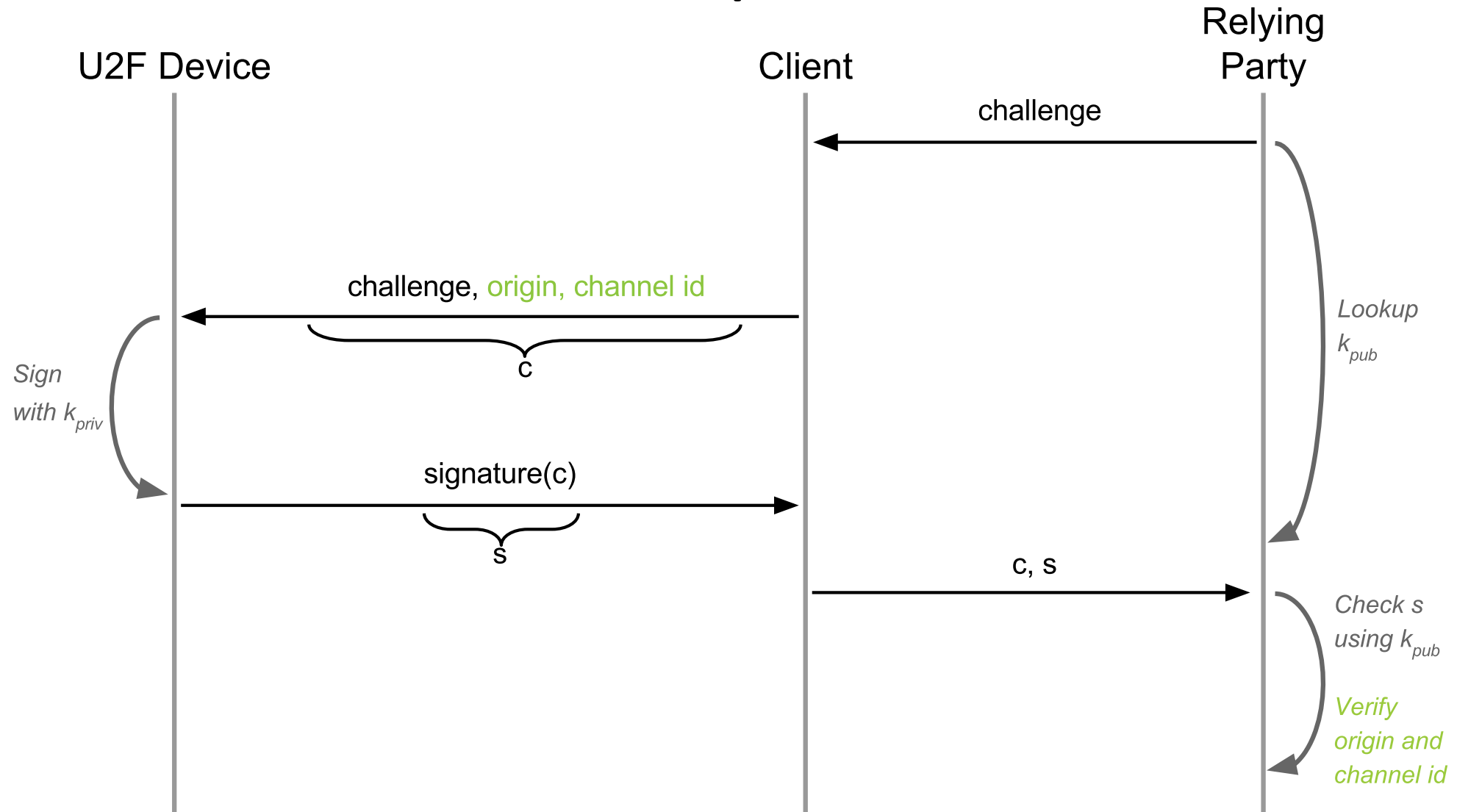
- What's the problem?
 - OTP not bound to the website it is sent to
 - OTP can be replayed
- What's the solution?
 - Bind the OTP to the specific website
 - Then OTP sent to bankofthev**v**est.com can't be sent to bankofthe**v**est.com (for example)

Universal second factor (U2F)

- Addresses OTP's weakness to MitM
- Website's *origin* is sent to the device and cryptographically bound to the response (next diagram)



U2F MitM protection



Disadvantages

- Token can be lost, stolen, or counterfeited
- Requires an individual physical token
- Requires an extra step (mildly inconvenient)
- Hardware can be expensive...
 - ...but usually isn't
 - \$20 for U2F key from Yubico
 - Google, Facebook, and Yubico were all giving these away at a recent conference I attended (pre-pandemic!)

WebAuthn (passkeys)

- Successor to U2F
- Similar to U2F there's a
 - Server (WebAuthn Relying Party)
 - Browser (WebAuthn Client)
 - Authenticator (the device in the U2F diagram 2 slides ago)
- Can be passwordless



User



User Agent

Web browser



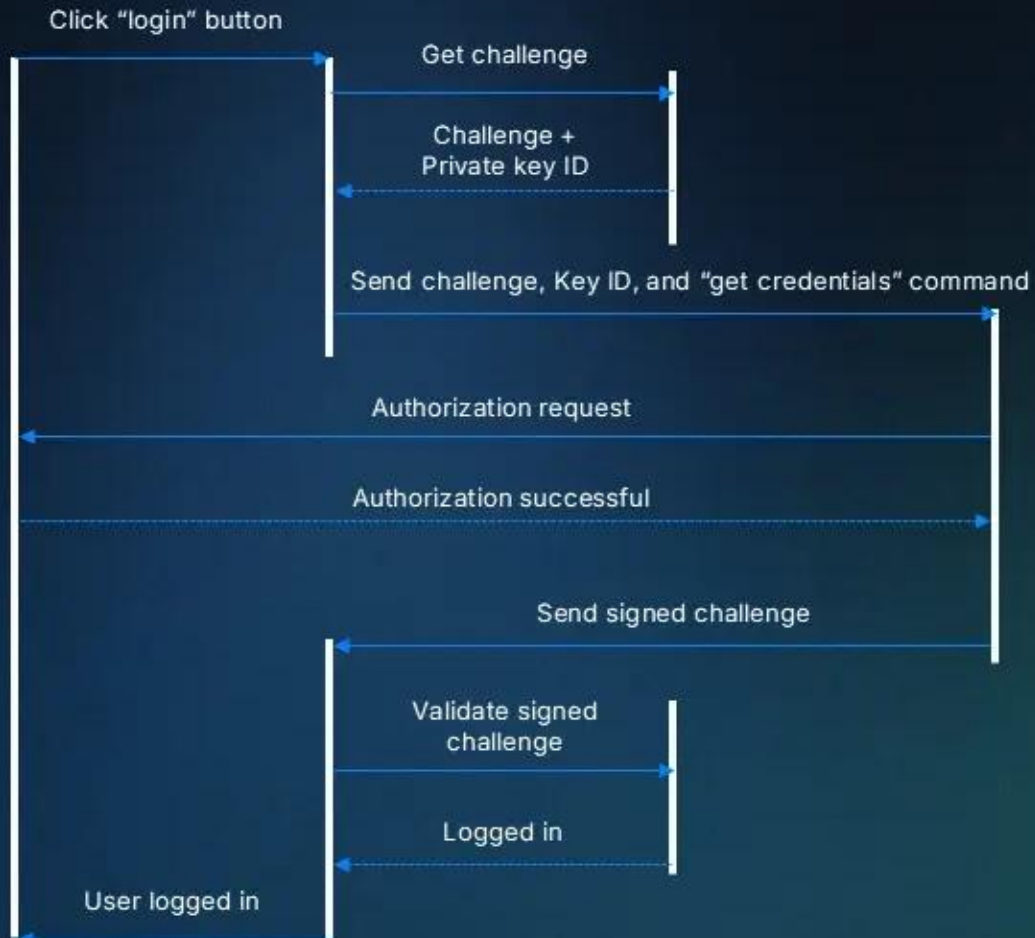
Relying Party

bakedpotato[.]com



Authenticator

Windows Hello, Touch ID, YubiKey, etc.

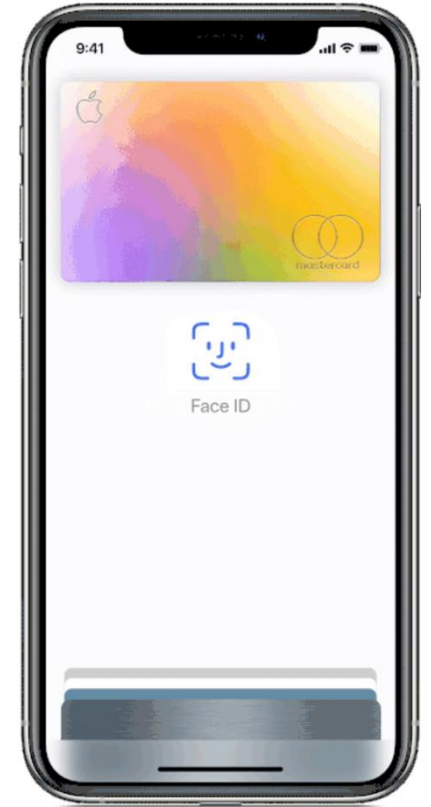


Authentication flow

- The authenticator verifies the user (finger print, facial recognition, pressing a button)
- The signed challenge is sent to the browser and from there to the server

Authenticator types

- Platform authenticators store cryptographic credentials in a Trusted Platform Module (TPM) and authenticate users, usually via biometrics
 - Apple's Face ID and Touch ID
 - Windows Hello
- Roaming authenticator, same idea but mobile
 - Security keys



https://developers.yubico.com/Developer_Program/WebAuthn_Starter_Kit/Platform_and_Roaming_Authenticators.html

<https://webauthn-doc.spomky-labs.com/webauthn-in-a-nutshell/authenticators>

BIOMETRIC AUTHENTICATION

Biometrics

- Something the user *is* or *does*
- Derive a signature from biological features of user
 - Voice, fingerprint, face, retina, handwriting, gait
- Advantages?
- Disadvantages?

Disadvantages

- Imprecise measurements require *approximate* matching
 - Essentially a machine learning task
 - False negatives *and* false positives have a cost
- Measurements change over time
- Poor accessibility
- Cannot be replaced or concealed
- Replay attacks/spoofing possible
- Can be legally compelled to provide biometrics (although this may be changing; United States v. Brown 2025)

OPM Breach

Krebs on Security

In-depth security news and investigation



[BLOG ADVERTISING](#)

[ABOUT THE AUTHOR](#)

Congressional Report Slams OPM on Data Breach

A massive data breach at the **U.S. Office of Personnel Management (OPM)** that exposed background investigations and fingerprint data on millions of Americans was the result of a cascading series of cybersecurity blunders from the agency's senior leadership on to the outdated technology used to secure the sensitive data, according to a lengthy report released today by a key government oversight panel.



My New Book!



Facial Recognition

[Browse Journals & Magazines](#) > [IEEE Transactions on Informat...](#) > [Volume: 9 Issue: 7](#) [?](#)

Spoofing Face Recognition With 3D Masks

Purchase or Sign In
to View Full Text

14
Paper
Citations

1588
Full
Text Views

Related Articles

Face
Verification
With Local
Sparse
Representation

3D Assisted
Face
Recognition:
Dealing With
Expres...

Depth
Estimation of
Face Images
Based on the
Cons...

2

Author(s)

▼ Nesli Erdogan ; ▼ Sébastien Marcel

[View All Authors](#)

Abstract

[Authors](#)

[Figures](#)

[References](#)

[Citations](#)

[Keywords](#)

[Metrics](#)

[Media](#)

Abstract:

Spoofing is the act of masquerading as a valid user by falsifying data to gain an illegitimate access. Vulnerability of recognition systems to spoofing attacks (presentation attacks) is still an open security issue in biometrics domain and among all biometric traits, face is exposed to the most serious threat, since it is particularly easy to access and reproduce. In this paper, many different types of face spoofing attacks have been examined and various algorithms have been proposed to detect them. Mainly focusing on 2D attacks forged by displaying printed photos or replaying recorded videos on mobile devices, a significant portion of these studies ground their arguments on the flatness of the spoofing material in front of the sensor. However, with the advancements in 3D reconstruction and printing technologies, this assumption can no longer be maintained. In this paper, we aim to inspect the spoofing potential of subject-specific 3D facial masks for different recognition systems and address the detection problem of this more complex attack type. In order to assess the spoofing performance of 3D masks against 2D, 2.5D, and 3D face recognition and to analyze various texture-based countermeasures using both 2D and 2.5D data, a parallel study with comprehensive experiments is performed on two data sets: the Morpho database which is not publicly available and the newly distributed 3D mask attack database.

OTHER SCHEMES

Multifactor Authentication

- Next level 2FA
- Combination of biometrics, knowledge, and possession

Behavior Profiling

- Track access behavior of users
 - Systems used
 - Times and locations when active
 - Typical usage
- Look for anomalous or fraudulent behavior
- Impossible travel: “Why is this person who was in Oberlin 2 minutes ago logging in from Prague?”
- Used in fraud prevention